



Master MIAE (apprentissage)

GraphBench Challenge

Réalisé du 7 mai 2026 au 12 mai 2026

Lien vers le dépôt GitHub : <https://github.com/4143/graphbench-project>

Nombre de conjectures réfutées : 89

Score final: 1344.14

Fait par

**Franck Zheng ,
Alex Brindusoiu,**

Chapitre 1

Introduction et objectif

Notre objectif est de concevoir en binôme la meilleure heuristique possible afin de réfuter des conjectures de manière générique, même si, pour ce faire, un jeu de données de 100 conjectures nous a été fourni.

Pour cela, nous utilisons une fonction de violation. Notre algorithme doit générer un contre-exemple, c'est-à-dire un graphe valide pour lequel l'objectif algorithmique est de maximiser la fonction de violation :

$$violation(G) = A(G) - f(B(G)) > 0$$

Bien sûr, la contrainte temporelle n'est pas notre seule contrainte ; il faut aussi respecter la classe du graphe.

Pour résoudre ce problème d'optimisation sous contrainte temporelle, notre approche se divise en deux étapes :

1. **Heuristique de recherche locale**
2. **Architecture FunSearch**

Nous automatisons l'amélioration de la fonction de score à l'aide d'un LLM (ici `gpt-4o-mini`). Le LLM génère du code Python pour proposer une fonction d'évaluation qui a un meilleur score :

$$F(G) = violation(G) + bonus(G) - penalty(G)$$

Notre programme produit à la fin, comme demandé dans la consigne, dans notre fichier de résultats :

- un graphe candidat,
- son encodage au format `graph6`,
- les valeurs des invariants nécessaires,
- une preuve calculatoire que la conjecture est violée.

Chapitre 2

Heuristique simple

On est partis d'une solution de base proposée par l'IA à laquelle on a apporté des modifications en fonction des problèmes que nous observions. Nous sommes partis des mutations possibles proposées et nous en avons ajouté certaines.

De plus, nous avons ajouté un fichier `graph_utils.py` qui contient des générateurs de graphes spécialisés qui sont utilisés si certaines stratégies sont reconnues ; il ne s'agit pas de graphes pré-enregistrés mais de manières de faire plus spécialisées. Ce mécanisme s'immisce dans la phase d'initialisation dès le début.

2.1 Fonction d'évaluation

Pour ce qui est de la fonction d'évaluation `heuristic_score`, cette fonction nous permet de savoir si on est proche du résultat par un score, contrairement au fait de dire simplement que c'est vrai ou faux. Cela nous permet d'améliorer l'heuristique de notre côté.

2.2 Recherche de contre-exemples

Pour ce qui est de la recherche de contre-exemples de on a la fonction `search_counterexample`. Les étapes que l'on applique sont :

1. Générer une population initiale.
2. Pour chaque candidat, on applique des mutations.
3. On garde les mutations avec les meilleurs score.
4. On retourne le résultat dès qu'un contre-exemple est trouvé.

Dans notre phase de sélection, nous avons appliqué différentes méthodes qui nous ont été proposées par l'IA (en dehors de `FunSearch`) comme :

2.3 L'apprentissage par Bandit Manchot (UCB1)

Cette méthode nous permet, par l'intermédiaire de la fonction `AdaptiveMutationSelector`, de garder en mémoire les mutations qui ont fait grimper le score d'où l'importance qu'il ne soit pas binaire.

Et de creuser de ce côté. Bien sûr, cette méthode ne bloque pas le processus de recherche dans une seule direction ; c'est là son avantage car elle permet quand même d'explorer d'autres directions si besoin.

2.4 Réparation

Pour finir, après chaque mutation, la fonction **repair** intervient pour s'assurer que le graphe mutant respecte toujours les règles de base de la classe de graphes testée.

Gestion de la stagnation Nous avons aussi ajouté le fait que si nous n'avons pas de résultat concluant au bout de 150 itérations, nous générons une nouvelle population (Restart) afin de ne pas perdre trop de temps sur un chemin inefficace.

Chapitre 3

Architecture FunSearch

Notre architecture est basée sur gpt-4o-mini. Nous avons aussi effectué des tentatives avec gemini-2.5-flash.

Nous avons choisi l'IA qui était le plus souvent disponible, dans notre cas ça a été gpt-4o. Car Gemini était souvent sollicité.

On va d'abord prendre un petit échantillon de conjectures de notre fichier benchmark sur lequel nous allons tester les fonctions de résolution proposées par le LLM. Nous ferons ensuite 3 tentatives dont nous prendrons la plus prometteuse en termes de score.

Il faut savoir qu'on lui passe aussi en argument de notre prompt toutes les variables disponibles, de plus on lui pré-calcule les invariants pour une meilleure compréhension. On lui donne aussi un exemple. Tout cela passe par l'intermédiaire d'une *seed*.

Chapitre 4

Résultats expérimentaux

```
=====
RÉSULTATS FINAUX
=====
Conjectures réfutées : 89 / 100
Score total          : 1344.14
Temps total réel     : 684.7s
Résultats sauvegardés: D:\Github\graphbench-project\src\..\results\results.json
=====

=== STATISTIQUES DES MUTATIONS ===
Total mutations: 1319245
mutate_contract_edge      : 116065 ( 8.8%)
mutate_add_path           : 114292 ( 8.7%)
mutate_add_pendant_path   : 114218 ( 8.7%)
mutate_duplicate_vertex   : 67695 ( 5.1%)
mutate_remove_vertex      : 65942 ( 5.0%)
mutate_add_edge           : 65522 ( 5.0%)
mutate_remove_edge        : 65399 ( 5.0%)
mutate_add_vertex         : 65293 ( 4.9%)
mutate_change_degree      : 65252 ( 4.9%)
mutate_rewire_edge        : 65246 ( 4.9%)
mutate_add_triangle       : 65232 ( 4.9%)
mutate_add_star           : 65166 ( 4.9%)
mutate_add_clique         : 65152 ( 4.9%)
mutate_swap_edges         : 65147 ( 4.9%)
mutate_remove_bridge      : 65128 ( 4.9%)
mutate_tree_remove_leaf   : 49392 ( 3.7%)
mutate_tree_add_leaf      : 49067 ( 3.7%)
mutate_tree_subdivide     : 49066 ( 3.7%)
mutate_claw_free_add_edge : 20585 ( 1.6%)
mutate_claw_free_add_clique : 20386 ( 1.5%)
PS D:\Github\graphbench-project>
```

FIGURE 4.1 – Résultats

```
=====
RÉSULTATS FINAUX
=====
Conjectures réfutées : 85 / 100
Score total          : 2554.33
Temps total réel     : 1954.9s
Résultats sauvegardés: D:\Github\graphbench-project\src\..\results\results.json
=====

=== STATISTIQUES DES MUTATIONS ===
Total mutations: 38196
mutate_contract_edge      : 3282 ( 8.6%)
mutate_duplicate_vertex   : 2913 ( 7.6%)
mutate_add_path           : 2895 ( 7.6%)
mutate_add_pendant_path   : 2872 ( 7.5%)
mutate_remove_vertex      : 2332 ( 6.1%)
mutate_add_edge           : 2196 ( 5.7%)
mutate_remove_edge        : 2166 ( 5.7%)
mutate_add_vertex         : 2098 ( 5.5%)
mutate_rewire_edge        : 2077 ( 5.4%)
mutate_add_triangle       : 2046 ( 5.4%)
mutate_remove_bridge      : 2028 ( 5.3%)
mutate_change_degree      : 2023 ( 5.3%)
mutate_swap_edges         : 2005 ( 5.2%)
mutate_add_star           : 2004 ( 5.2%)
mutate_add_clique         : 2001 ( 5.2%)
mutate_tree_remove_leaf   : 941 ( 2.5%)
mutate_tree_add_leaf      : 867 ( 2.3%)
mutate_tree_subdivide     : 856 ( 2.2%)
mutate_claw_free_add_edge : 298 ( 0.8%)
mutate_claw_free_add_clique : 296 ( 0.8%)
PS D:\Github\graphbench-project>
```

FIGURE 4.2 – Résultats FunSearch

Chapitre 5

Discussion scientifique

— Quelles conjectures sont faciles à réfuter ?

Dans l'ensemble ce sont les conjectures avec les invariants taille d'une clique maximum et degré moyen comme par exemple la 980 ou la 981. Car l'opération d'ajouter des arêtes est l'une des premières que l'on fait, c'est même la première qui nous est proposée dans la partie mutations possibles.

— Quels invariants semblent difficiles à manipuler ?

Les invariants les plus difficiles à manipuler sont ceux qui reposent sur une optimisation globale du graphe, notamment le nombre de domination, la taille d'un ensemble indépendant maximum, la taille d'une clique maximum et la taille d'une couverture minimum par sommets, ainsi que les invariants de distance globale comme le diamètre et le rayon.

À l'inverse, les invariants locaux comme les degrés ou le nombre d'arêtes sont beaucoup plus stables et faciles à manipuler pour obtenir un résultat sans tout casser par des mutations simples.

Après, cela est peut-être dû à notre approche car de par notre algorithme nous effectuons beaucoup de mutations mais qui apportent de très faibles modifications. Une heuristique qui travaillerait avec des mutations plus importantes dès le début pourrait régler notre problème pour les conjectures non réfutées, mais cela rentre en contradiction avec notre méthode initiale, ce qui aurait signifié tout recommencer sans être sûrs que ce soit la bonne approche.

— Quelles mutations sont efficaces ?

```

=== STATISTIQUES DES MUTATIONS ===
Total mutations: 1319245
mutate_contract_edge      : 116065 ( 8.8%)
mutate_add_path           : 114292 ( 8.7%)
mutate_add_pendant_path   : 114218 ( 8.7%)
mutate_duplicate_vertex   : 67695 ( 5.1%)
mutate_remove_vertex      : 65942 ( 5.0%)
mutate_add_edge           : 65522 ( 5.0%)
mutate_remove_edge        : 65399 ( 5.0%)
mutate_add_vertex         : 65293 ( 4.9%)
mutate_change_degree      : 65252 ( 4.9%)
mutate_rewire_edge        : 65246 ( 4.9%)
mutate_add_triangle       : 65232 ( 4.9%)
mutate_add_star           : 65166 ( 4.9%)
mutate_add_clique         : 65152 ( 4.9%)
mutate_swap_edges         : 65147 ( 4.9%)
mutate_remove_bridge      : 65128 ( 4.9%)
mutate_tree_remove_leaf   : 49392 ( 3.7%)
mutate_tree_add_leaf      : 49067 ( 3.7%)
mutate_tree_subdivide     : 49066 ( 3.7%)
mutate_claw_free_add_edge : 20585 ( 1.6%)
mutate_claw_free_add_clique : 20386 ( 1.5%)

```

FIGURE 5.1 – mutations les plus utilisées

— L’architecture FunSearch améliore-t-elle réellement l’heuristique initiale ?

```

=====
RÉSULTATS FINAUX
=====
Conjectures réfutées : 85 / 100
Score total          : 2554.33
Temps total réel     : 1954.9s
Résultats sauvegardés: D:\Github\graphbench-project\src\..\results\results.json
=====

=== STATISTIQUES DES MUTATIONS ===
Total mutations: 38196
mutate_contract_edge      : 3282 ( 8.6%)
mutate_duplicate_vertex   : 2913 ( 7.6%)
mutate_add_path           : 2895 ( 7.6%)
mutate_add_pendant_path   : 2872 ( 7.5%)
mutate_remove_vertex      : 2332 ( 6.1%)
mutate_add_edge           : 2196 ( 5.7%)
mutate_remove_edge        : 2166 ( 5.7%)
mutate_add_vertex         : 2098 ( 5.5%)
mutate_rewire_edge        : 2077 ( 5.4%)
mutate_add_triangle       : 2046 ( 5.4%)
mutate_remove_bridge      : 2028 ( 5.3%)
mutate_change_degree      : 2023 ( 5.3%)
mutate_swap_edges         : 2005 ( 5.2%)
mutate_add_star           : 2004 ( 5.2%)
mutate_add_clique         : 2001 ( 5.2%)
mutate_tree_remove_leaf   : 941 ( 2.5%)
mutate_tree_add_leaf      : 867 ( 2.3%)
mutate_tree_subdivide     : 856 ( 2.2%)
mutate_claw_free_add_edge : 298 ( 0.8%)
mutate_claw_free_add_clique : 296 ( 0.8%)
PS D:\Github\graphbench-project>

```

FIGURE 5.2 – Résultats FunSearch

De par nos résultats (le score dépend de la puissance du pc), nous n’avons pas observé d’améliorations significatives de par l’utilisation d’une architecture FunSearch. Nous sommes plus proches d’une régression. Mais ce qui est paradoxal c’est que nous avons un même score pour beaucoup moins de mutations.

— L’IA a-t-elle produit des idées utiles ou seulement du code ?

Pour le coup, oui, l’IA peut produire des idées utiles lorsque nous ne sommes pas experts du sujet ou, du moins, nous proposer des solutions existantes à étudier dont nous n’avions pas connaissance. Cependant, l’imbrication finale des différentes propositions reste le fruit d’un travail humain.

Malheureusement, sur la partie *FunSearch*, nous n’avons pas vraiment observé ce phénomène. Il faut préciser qu’il ne s’agissait que d’un simple test et que plusieurs itérations au sein d’une même conversation auraient pu donner

des résultats plus impressionnants. Nous avons d'ailleurs pu le constater lors de la **Phase 1**, quand nous concevions notre heuristique avec l'aide des LLM. prompte.